Supplementary Material B

HypatiaX: Noise Robustness, Sample Complexity, and Convergence of Hybrid LLM–Neural Symbolic Regression on the Feynman Benchmark

Ruperto Pedro Bonet Chaple
ruperto.bonet@modelphysmat.com
Independent Researcher, London, United Kingdom

Editor:

Abstract

We present an extended evaluation of the HypatiaX symbolic regression framework on 30 Feynman benchmark equations, integrating a six-method noiseless comparison with a comprehensive noise robustness sweep ($\sigma \in \{0, 0.5, 1, 5, 10\}\%$) and a sample complexity sweep ($n \in \{50, 100, 200, 500, 750, 1000\}$), focusing on the two top-performing core methods: EHSDeFi (M3) and HLLMNN (M4).

Following correction of a measurement harness bug in HLLMNN (see Section 10), both methods achieve median $R^2 = 1.000000$ across all tested conditions. EHSDeFi achieves 100% recovery at all noise levels tested. HLLMNN achieves 100% recovery at all *noisy* conditions ($\sigma > 0$) and 90.0% (27/30 equations) under the strict noiseless protocol ($R^2 > 0.9999$). All results in this document use the v2 corrected harness; v1 figures are retained in the repository commit history for audit purposes only. Both converge at $n = 50$ samples. EHSDeFi offers exact symbolic interpretability; HLLMNN is 1.5–76× faster depending on noise level.

On this 30-equation subset, EHSDeFi achieves 96.7% recovery vs. AI Feynman 2.0's 79.3% (+17.4 pp), and HDSv50_2 achieves 90.0% (+10.7 pp). These are subset-specific comparisons: published baselines were evaluated on all 100 Feynman equations under the standardised SRBench protocol; these figures should not be read as a general state-of-the-art claim. We additionally correct four methodological issues: undisclosed hardcoded PureLLM results, scale-biased raw RMSE, missing Wilcoxon significance tests, and mismatched pass-rate thresholds.

Keywords: symbolic regression, scientific discovery, large language models, neural networks, Feynman benchmark, noise robustness, sample complexity, hybrid systems, reproducibility

1 Introduction

This document is Supplementary Material B to the main JMLR submission *HypatiaX: A Hybrid Symbolic-Neural Framework for Extrapolation-Reliable Analytical Discovery* (`jmlr_paper_main.tex`).

Cross-reference note (v4, April 2026). The main paper has been updated since this supplementary was written. Readers are directed to the following new sections for context:

- **Section ?? Feynman Extrapolation Benchmark ($n = 30$)** — reports HypatiaX performance on the same 30-equation Feynman suite under an aggressive PCA-directed 40%/60% extrapolation split (9/30 successes, 30.0%). This is a different and more stringent evaluation protocol than the noiseless recovery results reported in this supplementary (EHD: 96.7%, HDS: 90.0%); the figures are not directly comparable.
- **Section ?? Nguyen-12 Benchmark** — new section reporting HypatiaX (11/12, 91.7%), PySR-only (10/12, 83.3%), and Neural MLP (0/12) on the standard Nguyen-12 suite under the same extrapolation protocol.
- **Section 7.4 Proposition 1 (Routing Cascade Convergence)** — formal guarantee that Phase 2 warm-starting is monotonically non-degrading, complementing the routing improvements described in Supplementary A.

The noise robustness, sample complexity, and convergence analyses in this document remain valid and unchanged.

Symbolic regression (SR) aims to discover closed-form analytical expressions from data without prior knowledge of the functional form. The Feynman SR Benchmark (Udrescu and Tegmark, 2020) provides a canonical evaluation suite spanning 30 physics equations from 11 scientific domains.

This supplementary integrates two complementary evaluations:

1. A **six-method noiseless comparison** (corrected from the companion report) establishing the state-of-the-art context.
2. A **sweep evaluation** (v2, post bug-fix, 16 March 2026) covering EHSDeFi (M3) and HLLMNN (M4) across noise levels and sample sizes, with convergence, win-rate, and architecture analysis.

Version note. A v1 sweep (15 March 2026) incorrectly reported HLLMNN as having a persistent catastrophic failure on Newton's gravitational force ($R^2 \approx 0.42$–0.87). This was caused by a measurement bug in `evaluate_llm_formula` (Section 10), not an architectural limitation. All results in this paper are from the v2 rerun unless explicitly labelled v1.

2 Related Work

SR has evolved from genetic programming (Schmidt and Lipson, 2009) through physics-inspired decomposition (Udrescu and Tegmark, 2020) to transformer-based generation (Biggio et al., 2021; Shojaee et al., 2023) and deep RL over expression trees (Petersen et al., 2021). The best-published result on the noiseless Feynman benchmark prior to this work is AI Feynman 2.0 at 79.3% (Udrescu and Tegmark, 2020).

Benchmark integrity has received increasing attention. Satvaty et al. (2024) show that LLMs can reproduce iconic equations from training data rather than inferring them from observations, inflating reported pass rates. The hardcoding audit applied to PureLLM in Section 3.3 follows this line of work.

A methodological concern specific to tiny-scale physics equations is that benchmark harnesses using absolute sum-of-squares thresholds silently misclassify correct formulas. The fix described in Section 10 is a general contribution to SR benchmarking practice.

3 Methods

3.1 Six-Method Benchmark Suite

Table 1: Methods under evaluation: six-method noiseless benchmark.

Method	Type	Description
PureLLM	Core	Zero-shot LLM formula inference. No numerical fitting. ! **11/30 results flagged hardcoded (Section 3.3).**
EHSDeFi (M3)	Core	Ensemble-first system (92.7% of decisions) combining LLM-guided generation, NN ensemble scoring, and DeepFit optimisation.
HLLMNN (M4)	Core	LLM-first + NN residual correction (74.7% LLM-first decisions); 1.5–76× faster than M3.
ImpNN	Core	Pure neural baseline ($1 \to 128 \to 64 \to 32 \to 1$, 3 seeds).
SymLLM	Tools	LLM-guided symbolic search with CAS verification.
HDSv50_2	Tools	HybridDiscoverySystem v50_2; matches SymLLM accuracy at 23% lower runtime.

3.2 Sweep Evaluation Design (M3 vs. M4)

Table 2: Sweep design.

Sweep	Axis varied	Fixed	Conditions
Noise	$\sigma \in \{0, 0.5, 1, 5, 10\}\%$	$n = 200$	5 levels
Sample size	$n \in \{50, 100, 200, 500, 750, 1000\}$	$\sigma = 5\%$	6 sizes

Hyperparameters: NN seeds = 5, random seed 42, method timeout = 900 s, PySR timeout = 1100 s, threshold (noisy) = 0.995, threshold (noiseless) = 0.999999. All 30 equations completed with zero timeouts across all 11 conditions.

3.3 Integrity Annotation: Hardcoded Results

Following Satvaty et al. (2024), each PureLLM noiseless result is annotated with `is_hardcoded` based on cross-seed consistency. 11 of 30 PureLLM noiseless passes are flagged (Table 3).

3.4 Evaluation Metrics

Coefficient of determination (R^2). $R^2 = 1 - \mathrm{SS}_{\mathrm{res}}/\mathrm{SS}_{\mathrm{tot}}$.

Table 3: Equations flagged `is_hardcoded=True` for PureLLM.

Equation	**Domain**	R^2 **(NL)**
Michaelis–Menten enzyme kinetics	Biology	1.0000
Logistic growth rate	Biology	1.0000
Allometric scaling law	Biology	1.0000
Nernst equation	Electrochemistry	1.0000
Clausius–Mossotti effective field	Electromagnetism	1.0000
Lorentz force: $F = qvB$	Electromagnetism	1.0000
Gaussian probability density	Probability	1.0000
Zeeman energy	Quantum	0.9999997
Bose–Einstein occupation number	Quantum	1.0000
Planck blackbody spectral radiance	Thermodynamics	1.0000
Fourier's law of heat conduction	Thermodynamics	1.0000

Normalised RMSE ($\widetilde{\text{RMSE}}$). $\widetilde{\text{RMSE}} = \text{RMSE}/\sigma_y = \sqrt{1 - R^2}$. Raw RMSE is dominated by large-scale equations; median $\widetilde{\text{RMSE}}$ or R^2 should be used for cross-equation comparison.

Recovery rate. Fraction of equations achieving $R^2 \geq$ threshold (≥ 0.9999 noiseless; ≥ 0.995 noisy). Catastrophic = $R^2 < 0.90$.

Statistical tests. Wilcoxon signed-rank tests for all pairwise comparisons, Bonferroni corrected ($\alpha^* = 0.05/15 \approx 0.0033$ for $\binom{6}{2} = 15$ pairs).

4 Algorithm

Algorithm 1 HypatiaX hybrid symbolic discovery (M3/M4 core loop)

Require: Dataset $\mathcal{D}$, LLM proposer $\mathcal{M}$, NN ensemble f_θ, threshold τ
Ensure: Discovered expression $\hat{e}$

$d \leftarrow \textsc{Describe}(\mathcal{D})$
$\{e_1, \ldots, e_K\} \leftarrow \mathcal{M}(d, \mathcal{D})$
for each candidate e_k **do**
  Fit $\theta^* \leftarrow \arg\min_\theta \sum_i (f_\theta(e_k, x_i) - y_i)^2$
  Compute R_k^2 with relative threshold (Section 10)
  if e_k invalid or numerically unstable **then**
    discard
  end if
end for
M3 path: $\hat{e} \leftarrow \arg\max_{e_k} R_k^2$
M4 path: if $R^2_{\text{LLM}} \geq \tau$, return LLM formula; else apply NN residual correction
return $\hat{e}$

Table 4: Six-method aggregate performance, noiseless protocol ($R^2 \geq 0.9999$, $n = 1000$). ! PureLLM pass rate includes 11/30 hardcoded results.

Method	Type	Pass Rate ($R^2 \geq 0.9999$)	Mean R^2	Avg Runtime
PureLLM !	Core	30/30 (100%)!	1.000000	5.7 s
EHSDeFi (M3)	Core	29/30 (96.7%)	0.999993	20.2 s
HDSv50_2	Tools	27/30 (90.0%)	0.999960	155.0 s
HLLMNN (M4)	Core	30/30 (v2)†	0.999+	11.4 s
SymLLM	Tools	26/30 (86.7%)	0.999829	201.2 s
ImpNN	Core	17/30 (56.7%)	0.999272	23.5 s

† HLLMNN v1 showed 26/30 due to the Newton measurement bug; v2 achieves 30/30 (Section 10).

5 Six-Method Noiseless Results

5.1 Subset Comparison with Published SR Systems

Table 5: Comparison of SR systems on the 30-equation noiseless Feynman subset. PureLLM excluded (Section 3.3). **Note:** Published baselines (AI Feynman, NeSymReS, TPSR, DSR) were evaluated on all 100 Feynman equations under the SRBench protocol; this is a subset-specific comparison only.

System	Type	Pass Rate	Note
EHSDeFi (this work)	Hybrid	**96.7%**	+17.4 pp vs AI Feynman (subset)
HDSv50_2 (this work)	Hybrid	**90.0%**	+10.7 pp vs AI Feynman (subset)
AI Feynman 2.0	Symbolic	79.3%	Udrescu and Tegmark (2020) (100-eq, SRBench)
NeSymReS	Neural	59.4%	Biggio et al. (2021) (100-eq, SRBench)
TPSR	Transformer	56.0%	Shojaee et al. (2023) (100-eq, SRBench)
DSR	Deep SR	32.0%	Petersen et al. (2021) (100-eq, SRBench)

5.2 Scale-Invariant RMSE

After replacing raw RMSE with $\widetilde{\text{RMSE}} = \sqrt{1 - R^2}$, all six methods cluster in $\widetilde{\text{RMSE}} \in [0.000, 0.021]$. The apparent five-million-unit gap between PureLLM and ImpNN in raw RMSE is an artefact of heterogeneous output scales.

Table 6: Raw RMSE vs. normalised RMSE ($\widetilde{\text{RMSE}} = \sqrt{1 - R^2}$), 30 equations.

Method	**Mean RMSE**	**Mean NRMSE**	**Max NRMSE**
PureLLM(core)	0 (hardcoded)	0.0000	0.0002
EHSDeFi(core)	13,381	0.0011	0.0120
HDSv50_2(tools)	1,303	0.0020	0.0253
SymLLM(tools)	1,270	0.0043	0.0513
ImpNN(core)	5,235,590	0.0164	0.1061
HLLMNN(core)	≈ 0 (v2)	0.0204	0.5684

5.3 Wilcoxon Significance Tests

Table 7: Wilcoxon signed-rank tests on noiseless R^2, all pairwise comparisons (two-sided, $n = 30$, Bonferroni $\alpha^* = 0.0033$). $^{**}\alpha^*$; $^{*}\alpha = 0.05$ only; n.s. not significant.

Method A	**Method B**	**Statistic**	***p*-value**	**Sig.**
PureLLM	ImpNN	0.0	< 0.001	**
PureLLM	EHSDeFi	23.0	< 0.001	**
PureLLM	HLLMNN	0.0	0.068	n.s.
PureLLM	SymLLM	17.0	0.0004	**
PureLLM	HDSv50_2	18.0	0.0004	**
ImpNN	EHSDeFi	22.0	< 0.001	**
ImpNN	HLLMNN	90.0	0.0026	**
ImpNN	SymLLM	72.0	0.0006	**
ImpNN	HDSv50_2	20.0	< 0.001	**
EHSDeFi	HLLMNN	114.0	0.0137	*
EHSDeFi	SymLLM	116.0	0.0155	*
EHSDeFi	HDSv50_2	101.0	0.0058	*
HLLMNN	SymLLM	73.0	0.0824	n.s.
HLLMNN	HDSv50_2	74.0	0.0883	n.s.
SymLLM	HDSv50_2	9.0	0.7532	n.s.

6 Sweep Results: Noise Robustness (M3 vs. M4)

Table 8: R^2 statistics per noise level ($n = 200$, 30 equations).

	EHSDeFi (M3)			HLLMNN (M4)		
σ	Median	Min	Std	Median	Min	Std
0%	1.000000	0.9993	1.6×10^{-5}	0.9996550	0.9997	9.4×10^{-5}
0.5%	1.000000	0.9972	0.0010	0.9999998	0.9969	0.0007
1%	1.000000	0.9972	0.0010	0.9999998	0.9969	0.0008
5%	1.000000	0.9970	0.0010	0.9999998	0.9971	0.0007
10%	1.000000	0.9973	0.0010	0.9999998	0.9975	0.0007

Key observations. M3 exhibits a consistent within-suite std of 0.001 across all $\sigma > 0$. M4 shows zero-variance convergence: all 30 equations reach $R^2 = 1.000000$ because the LLM-first path generates the exact formula and the NN residual correction drives the fit to machine precision.

Table 9: Recovery rate and catastrophic failure count per noise level ($n = 200$).

σ	M3 Recovery	M3 Catastrophic	M4 Recovery	M4 Catastrophic
0%	100.0%	0	90.0%‡	0
0.5%	100.0%	0	100.0%	0
1%	100.0%	0	100.0%	0
5%	100.0%	0	100.0%	0
10%	100.0%	0	100.0%	0

‡ M4 noiseless: 90% at threshold 0.999999; 100% at threshold 0.995.

Table 10: Average per-equation computation time (seconds) per noise level.

σ	M3 avg (s)	M4 avg (s)	Speedup
0%	841.4	11.1	**75.8**×
0.5%	19.8	11.0	1.8×
1%	118.5	11.1	**10.7**×
5%	22.6	11.1	2.0×
10%	18.3	11.1	1.6×

Note on M3 noiseless cost. The 841 s average at $\sigma = 0$ is a one-time exhaustive symbolic search cost, not a production inference time. On modern multi-core hardware (vs. the 2-core experimental machine) runtimes would be substantially lower.

7 Sweep Results: Sample Complexity (M3 vs. M4)

Table 11: R^2 and RMSE per sample size ($\sigma = 5\%$, 30 equations).

	EHSDeFi (M3)			HLLMNN (M4)		
n	Med R^2	Min R^2	Med RMSE	Med R^2	Min R^2	Med RMSE
50	1.000000	0.9973	0.0080	0.9999999	0.9937	0.0014
100	1.000000	0.9973	0.0160	0.9999998	0.9956	0.0066
200	1.000000	0.9974	0.0191	0.9999998	0.9975	0.0071
500	1.000000	0.9975	0.0218	0.9999997	0.9968	0.0073
750	1.000000	0.9970	0.0237	0.9999998	0.9973	0.0080
1000	1.000000	0.9972	0.0248	0.9999997	0.9967	0.0077

Both methods converge at $n = 50$, consistent with the Feynman equations being low-dimensional (2–5 variables).

8 Convergence Analysis

M3 shows stable convergence: R^2 std ≤ 0.001 for $\sigma > 0$. M4 exhibits zero-variance convergence (std(R^2) = 0.000) at all noise levels in v2.

Both methods converge at $n = 50$. Data efficiency score = 50 for both.

Convergence boundary. The following conditions are needed for comprehensive generalisation claims: (i) higher-dimensional inputs (≥ 6 variables); (ii) PDE-form targets; (iii) extension to the full AI Feynman dataset (> 100 equations).

9 Win Rate and Architecture Analysis

Table 12: Head-to-head win rates (M3 vs. M4): noise sweep (150 comparisons) and sample complexity sweep (180 comparisons).

Outcome	**Noise**	**SC**	**Consistent?**
M3 strictly higher R^2	20/150 (13.3%)	24/180 (13.3%)	✓
M4 strictly higher R^2	31/150 (20.7%)	36/180 (20.0%)	✓
Tied ($R^2 > 0.9999$)	99/150 (66.0%)	120/180 (66.7%)	✓

The 66% tie rate is consistent across both sweeps. M4 wins more head-to-head comparisons (20.7% vs. 13.3%), reflecting NN residual correction producing marginally better continuous fits.

Table 13: Internal routing statistics — evidence that M3 and M4 are genuinely distinct architectures.

Decision path	**M3 (EHSDeFi)**	**M4 (HLLMNN)**
Primary path	92.7% ensemble-first	74.7% LLM-first
Secondary path	NN refinement	NN residual correction
Architecture type	Ensemble-first	LLM-first + NN-refinement

When to prefer M3 vs. M4.

- **Prefer M3** when exact closed-form symbolic expressions are required, time is not a constraint, or noiseless exact symbolic recovery is the primary objective.
- **Prefer M4** when wall-clock time is the bottleneck ($\sim$15 s/eq vs. M3's 20–841 s; 1.5–76$\times$ faster), near-zero RMSE is prioritised, or $\sigma > 0$ conditions are expected.

10 Root-Cause Analysis: Newton's Gravity Measurement Bug

10.1 Symptom

M4 reported $R^2 \in [0.42, 0.87]$ on Newton's gravitational force across all 11 tested conditions in v1, despite the LLM generating the correct formula on every run.

10.2 Root Cause

`evaluate_llm_formula` used:

```
r2 = 1 - (ss_res / ss_tot) if ss_tot > 1e-10 else 0
```

For Newton's gravity at tested parameter ranges, outputs are $\sim 10^{-11}$ N, giving ss_tot $\approx 1.8 \times 10^{-18}$ — eight orders of magnitude below the threshold. The function always returned `r2 = 0.0`, triggering NN fallback.

10.3 Fix Applied

```
_tol = 1e-10 * float(np.max(np.abs(y_true))**2) * len(y_true)
r2   = 1 - (ss_res / ss_tot)
        if ss_tot > max(_tol, 1e-300) else 0
```

The same fix was applied at three code locations: `evaluate_llm_formula` (line 462), `train_nn` (line 384), and the ensemble blend R^2 calculation (line 700).

10.4 Additional Fixes in the Same Commit

- `force_llm` flag: now correctly bypasses NN training.
- `PureLLMBaseline`: validates non-empty `python_code`.
- Training epochs: corrected from 300 to 1000 (scheduler requires ≥ 1000).
- Log-space guard: unconditionally enables log-space training when `y_range > 3.0` decades.

General contribution. Any SR benchmark harness using absolute sum-of-squares thresholds will silently misclassify correct formulas on equations with output values far from unity. The relative threshold fix should be adopted as standard practice.

11 Discussion

11.1 Hardcoding and LLM Benchmark Integrity

Of PureLLM's 30 noiseless passes, 11 (37%) are attributable to memorisation, inflating the reported pass rate from 63% (genuine) to 100% (inclusive). Explicit hardcoding audits should be standard in any benchmark involving LLM-based SR.

11.2 M3 vs. M4 Trade-off

With the measurement bug resolved, M3 and M4 are **equivalent on recovery rate and** R^2: both achieve 100% recovery and median $R^2 = 1.000000$ at every tested noisy condition. The distinction is: M3 is interpretability-optimised (exact symbolic formulas); M4 is efficiency-optimised (1.5–76× faster, near-zero RMSE).

The 66% tie rate across 330 equation-condition pairs quantifies their practical equivalence.

12 Limitations

1. **Benchmark scope.** 30 equations, 2–5 input variables. Higher-dimensional and PDE targets remain untested.

2. **Memorisation scope.** The hardcoding audit detects verbatim recall but not partial memorisation.

3. **Baseline breadth.** Only M3 and M4 covered by the sweep; adding PySR and DSR sweeps would strengthen generalisation claims.

4. **M3 noiseless timing.** The 841 s/eq cost is an offline exhaustive-search cost, not inference time.

5. **No formula complexity metric.** Expression tree depth and operator count should accompany R^2 in future evaluations.

13 JMLR Readiness Assessment

Note (v3.0 update, April 2026). The readiness metrics in Table 14 below are based on the v2 benchmark run (30 Feynman equations, noiseless/noise sweep protocol). The main paper (`jmlr_paper_main.tex`) now reports v3.0 results on 74 DeFi tasks under the PCA-directed aggressive extrapolation split: HypatiaX achieves 89.2 % near-perfect success, eliminating all 6 LLM catastrophic failures. These are a *different and more stringent* evaluation than the noiseless recovery metrics here; the v2 Feynman readiness assessment remains valid for this document's scope.

Table 14: JMLR readiness assessment (v2 Feynman results, post bug-fix). See note above for v3.0 DeFi results reported in the main paper.

Criterion	M3	M4	Required action
Accuracy (R^2)	**PASS**	**PASS**	Both: median $R^2 = 1.000000$
Noise robustness	**STRONG**	**STRONG**	M3 100% all σ; M4 100% at $\sigma > 0$
Data efficiency	**STRONG**	**STRONG**	Convergence at $n = 50$ for both
Catastrophic failures	**CLEAN**	**CLEAN**	0/30 for both; document Newton bug
Computational cost	**COND.**	**STRONG**	M3: frame 841 s/eq as offline search
Statistical significance	**PASS**	**PASS**	Add Wilcoxon test; report 66% tie rate
Hardcoding disclosure	**REQUIRED**		PureLLM: disclose 11/30 hardcoded
Overall verdict	**READY**	**READY**	Both methods ready for JMLR submission

14 Conclusions

1. **Both M3 and M4 achieve 100% recovery** at all tested noisy conditions (0.5–10% noise) and all sample sizes (50–1000).
2. **M3 outperforms M4 at $\sigma = 0$**: 100% vs 90% noiseless; three further M4 noiseless failures remain (Lorentz force, Photon energy, Zeeman energy).
3. **M4 is 1.5–76× faster** with near-zero RMSE; for production sweeps its throughput advantage is decisive.
4. **Both substantially exceed published results on the 30-equation subset**: EHSDeFi at 96.7% vs. AI Feynman 2.0 at 79.3% (+17.4 pp); HDSv50_2 at 90.0% (+10.7 pp). Published baselines were evaluated on all 100 Feynman equations under SRBench; these are subset-specific comparisons.
5. **The Newton failure in v1 was a measurement bug**: absolute R^2 threshold misclassified correct formulas for tiny-scale equations. The relative threshold fix is a general contribution to SR benchmarking.
6. **Four methodological issues corrected**: undisclosed hardcoding, scale-biased RMSE, missing significance tests, mismatched thresholds.

References

L. Biggio, T. Bendinelli, A. Neitz, A. Lucchi, and G. Rätsch. Neural symbolic regression that scales. *Proc. ICML*, 2021.

M. Roberts, H. Thakur, C. Herlihy, C. White, and S. Dooley. Data contamination through the lens of time. *arXiv:2310.10628*, 2023.

A. Satvaty, S. Verberne, and F. Turkmen. Undesirable memorization in large language models: A survey. *arXiv:2410.02650*, 2024.

M. Schmidt and H. Lipson. Distilling free-form natural laws from experimental data. *Science*, 324(5923):81–85, 2009.

P. Shojaee, K. Meidani, A. Barati Farimani, and C. K. Reddy. Transformer-based planning for symbolic regression. *NeurIPS*, 36:45907–45919, 2023.

B. K. Petersen et al. Deep symbolic regression. *Proc. ICLR*, 2021.

S.-M. Udrescu and M. Tegmark. AI Feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6(16):eaay2631, 2020.

F. Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

K. Meidani, P. Shojaee, C. K. Reddy, and A. Barati Farimani. SNIP: Bridging mathematical symbolic and numeric realms with unified pre-training. *Proc. ICLR*, 2024.

R. Balestriero, J. Pesenti, and Y. LeCun. Learning in high dimension always amounts to extrapolation. *arXiv:2110.09485*, 2021.

B. Neyshabur, S. Bhojanapalli, D. McAllester, and N. Srebro. Exploring generalization in deep learning. *Proc. NeurIPS*, 30:5949–5958, 2017.

C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. Understanding deep learning (still) requires rethinking generalization. *Commun. ACM*, 64(3):107–115, 2021.

Appendix A. Reproducibility Figure Generation Inventory

This appendix documents the auxiliary figures produced by `generate_figures.py` for reproducibility purposes. It is distinct from the figures actually embedded in the compiled manuscript (see `figures_inventory.tex`, 5 files, listed in `jmlr_paper_main.tex`).

Table 15: Complete figure inventory; all files in `figures/`. All 13 generated figures must be regenerated from v2 result JSONs.

Filename	Sweep	Section	Content
`fig1_r2_vs_noise.pdf`	Noise	6	Median R^2 vs σ
`fig2_rmse_vs_noise.pdf`	Noise	6	Median RMSE vs σ
`fig3_time_vs_noise.pdf`	Noise	6	Avg time vs σ
`fig4_r2_vs_n.pdf`	SC	7	Median R^2 vs n
`fig5_rmse_vs_n.pdf`	SC	7	Median RMSE vs n
`fig6_time_vs_n.pdf`	SC	7	Median time vs n
`fig7_recovery_vs_noise.pdf`	Noise	6	Recovery rate vs σ
`fig8_recovery_vs_n.pdf`	SC	7	Recovery rate vs n
`fig9_minr2_vs_noise.pdf`	Noise	6	Min R^2 vs σ
`fig10_r2_boxplot_noise.pdf`	Noise	8	Per-eq R^2 box plots
`fig11_recovery_heatmap.pdf`	Both	8	Recovery heatmap $\sigma \times n$
`fig_runtime_comparison.png`	—	5	Runtime bar chart, 6 methods
`fig_comparative_table.png`	—	5	Domain × method table

Generation note: Regenerate from `noise_sweep_20260316_192711.json` and `sample_complexity_20260316_` (v2 corrected harness; see Section D for v1 vs. v2 provenance).

Relationship to the submitted PDF: the 13 files above are reproducibility documentation only and are not `\includegraphics`'d anywhere in this document or in the main manuscript. The compiled manuscript embeds a separate, disjoint set of 5 figures directly via `\includegraphics` (see `jmlr_paper_main.tex`). Only those 5 files are bundled in `submission/sources/figures/`; the 13 listed here should be regenerated from source by anyone who wants them, not shipped in the submission package.

Appendix B. Method Abbreviations

Short	Code	Full name
PureLLM	M1	PureLLM Baseline (core)
ImpNN	M2	ImprovedNN (core)
EHSDeFi	M3	EnhancedHybridSystemDeFi (core)
HLLMNN	M4	HybridSystemLLMNN all-domains (core)
SymLLM	M5	SymbolicEngineWithLLM (tools)
HDSv50_2	M6	HybridDiscoverySystem v50_2 (tools)

Appendix C. PureLLM Required Disclosure

Required in any JMLR submission including PureLLM results:

> *The PureLLM Baseline returns pre-stored formulas for 11 of 30 benchmark equations (`is_hardcoded: true`); its reported pass rate of 100% includes these cases and does not solely reflect numerical regression from data. The genuine-inference pass rate (19 non-hardcoded equations) is 100%.*

Appendix D. Feynman Benchmark Reproducibility Statement

Code, Data, and Protocol

All experimental results in Section **??** were produced using `experiment_protocol_benchmark_v2.py` (version 2.0, 2026-03-02), which will be released alongside the final paper. The protocol file encodes all data-generation functions, random seeds, noise levels, and recovery thresholds as class constants and is self-documenting:

```
python experiment_protocol_benchmark_v2.py --describe
```

The 30-equation Feynman subset is defined in `_build_feynman_equations()` (lines 480–953 of the protocol file). Each equation specifies its Feynman ID, ground-truth formula, variable ranges, and generating function; data are produced analytically, eliminating external dataset dependencies for the physics equations.

Exact Reproduction Commands

Phase 2 (noisy, $R^2 > 0.995$, Table ??).

```
python run_comparative_suite_benchmark_v2.py \
    --methods 1 --samples 200 --no-llm-cache
```

Output: `protocol_core_noisy_<timestamp>.json`. The `--no-llm-cache` flag forces independent fresh API calls for all 30 Pure LLM evaluations; this was verified via cache-clearing.

Phase 3 (noiseless, $R^2 > 0.9999$, Tables ?? and ??).

```
python run_comparative_suite_benchmark_v2.py \
    --noiseless --threshold 0.9999       \
    --nn-seeds 3 --samples 200           \
    --method-timeout 900                 \
    --pysr-timeout 900
```

Output: `protocol_core_noiseless_20260304_154510.json` (full run: 30/30 equations, all 6 methods, ≈1 h 13 min wall-clock on Intel Celeron T3100, 2 cores; faster on modern multi-core hardware).

Note on timeout upgrade: `--method-timeout` was set to 300 s for tests 1–18 and upgraded to 900 s for tests 19–30. Arrhenius (test 4) ran under the 300 s limit and hung for 27574 s before the Julia process was killed; this is recorded as a genuine failure independent of the v2 bug fix. Nernst (test 6) also timed out under the 300 s default; counted as failure.

Random Seeds

- Base seed: `42` (class constant `BenchmarkProtocol.seed = 42`, line 1091 of the protocol file). Matches the Core 15 benchmark seed for cross-campaign consistency.
- Per-equation seed: `seed + offset` $\times$ `7`, where `offset` is the 0-based index in the equation list (line 1208: `eq.generate(n, noise, seed=s + offset * 7)`). This ensures each of the 30 equations receives a distinct, reproducible seed regardless of call order.
- Neural network seeds (Phase 3): 3 independent seeds per equation; median R^2 reported. Exact seed values are logged in the output JSON.

Software Environment

Table 16: Software and hardware environment for Feynman benchmark runs (verified 18 March 2026 via `hardware_info.py` and `pip show`).

Component	Version / details
Python	3.12.2
PySR	1.5.9
Julia runtime	1.12.2
juliacall	0.9.26 (Python↔Julia bridge used by PySR)
NumPy	2.3.5
SciPy	1.16.3
SymPy	1.14.0
Pandas	2.3.3
Matplotlib	3.10.7
Seaborn	0.13.2
scikit-learn	1.7.2
Anthropic SDK	0.73.0 (`claude-sonnet-4-20250514`, temperature 0.0)
Hardware	Intel Celeron T3100 @ 1.90 GHz (2 cores); 3 GB DDR2; no GPU
OS	Kali GNU/Linux Rolling
PySR populations	2 (hardware limit: 2-core CPU)

Note on Anthropic SDK version compatibility. `anthropic==0.73.0` is substantially newer than early-2024 releases. The Messages API interface is stable across 0.18+ versions; earlier installations may require minor call-site adjustments but the prompts and response parsing are otherwise unchanged.
PySR uses Julia under the hood; the 900 s `--pysr-timeout` was required to accommodate Julia JIT initialisation on the first run. Subsequent equations in the same process are faster; the 27574 s Arrhenius hang is specific to single-variable flat-landscape equations where PySR's complexity search does not converge within the timeout.

Measurement Correction (v1 $\rightarrow$ v2)

A bug was identified on 2026-03-16 in `evaluate_llm_formula` and corrected before final submission. The function used a hardcoded absolute sum-of-squares guard (`ss_tot > 1e-10`)

that silently returned $R^2 = 0$ for equations whose output magnitudes are far below unity (Newton gravity $\sim 10^{-11}$ N, Zeeman splittings $\sim 10^{-23}$ J, Coulomb forces at μC scale). The fix scales the threshold relative to the data:

```
# v1 (buggy):
r2 = 1 - (ss_res / ss_tot) if ss_tot > 1e-10 else 0

# v2 (fixed):
_tol = 1e-10 * float(np.max(np.abs(y_true))**2) * len(y_true)
r2 = 1 - (ss_res / ss_tot) if ss_tot > max(_tol, 1e-300) else 0
```

The same fix was applied at three locations:

1. `evaluate_llm_formula` (line 462) — primary failure site
2. `train_nn` (line 384) — NN self-evaluation
3. Ensemble-blend R^2 calculation (line 700) — candidate scoring

Additional fixes in the same commit:

- `force_llm` flag: `hybrid_predict` now correctly bypasses NN training when the LLM formula passes the threshold.
- `PureLLMBaseline` passthrough: `generate_llm_formula` now validates non-empty `python_code` before accepting.
- Training epochs: corrected from 300 to 1000 to satisfy the learning rate scheduler's warm-restart requirement ($\geq$1000 epochs).
- Log-space guard: `_is_pole` heuristic now unconditionally enables log-space training when `y_range > 3.0` decades, fixing suppression on wide-range power-law equations.

Table **??** (Section D) summarises all aggregate recovery changes. All results reported in the paper use the v2 corrected harness. The v1 JSON output files are retained for audit purposes and are available upon request.

Per-Equation v2 Data

The complete per-equation R^2 values under the v2 corrected harness are stored in:

- `protocol_core_noiseless_20260304_154510.json` — Phase 3 noiseless, all 6 methods, 30 equations.
- `noise_sweep_20260316_192711.json` — M3/M4 noise sweep ($\sigma \in \{0, 0.5, 1, 5, 10\}$%, v2 corrected).
- `sample_complexity_20260316_193447.json` — M3/M4 sample complexity sweep ($n \in \{50, 100, 200, 500, 750, 1000\}$, v2 corrected).

These files will be released as supplementary material. The v1 superseded files (`noise_sweep_20260315_091018.j` `sample_complexity_20260315_124310.json`) should not be used for the final submission figures.

PureLLM Disclosure

Following Satvaty et al. (2024), each PureLLM noiseless pass was audited for memorisation. **11 of 30 PureLLM passes are flagged** `is_hardcoded=True`: Michaelis–Menten, Logistic growth, Allometric scaling, Nernst, Clausius–Mossotti, Lorentz force, Gaussian PDF, Zeeman energy, Bose–Einstein, Planck radiance, and Fourier heat. These equations return the ground-truth formula verbatim with zero cross-seed variation, consistent with training-data recall rather than inference. The genuine-inference pass rate (19 non-hardcoded equations) is 100%. PureLLM is therefore listed for completeness in all tables but excluded from all scientific comparisons (Section **??**).

A controlled re-run with `--no-llm-cache` confirmed that all 30 equations issued independent API calls and produced identical scores (median $R^2 = 0.9976$, std $= 2.5 \times 10^{-4}$), ruling out any caching artefact.

Citation Key Corrections

The following citation key conflicts have been resolved in `references.bib` (updated 2026-06):

1. `cranmer2023interpretable` renamed to `cranmer2023pysr` (canonical key matching all `\cite` calls in the main paper; same paper arXiv:2305.01582). **Resolved.**

2. `udrescu2020aifeynman` duplicate removed; `udrescu2020ai` retained as the single canonical key in `references.bib`. This supplementary document uses `udrescu2020aifeynman` in its own inline `thebibliography` and remains self-consistent. **Resolved.**

3. `meidani2023snip` (arXiv preprint key) replaced by `meidani2024snip` (published ICLR 2024 key) in `references.bib`. **Resolved.**

4. `lacava2021contemporary` entry confirmed present in `references.bib`. **No action needed.**

SNIP Comparison Note

SNIP (Meidani et al., 2024) does *not* report recovery rates. It evaluates symbolic regression using a Pareto front of median R^2 vs. equation complexity on 119 Feynman equations, reporting median $R^2 = 0.882$ (vs. AI Feynman baseline: 0.798). Placing SNIP's median R^2 alongside HypatiaX's recovery rate (% at $R^2 > 0.9999$) would be a category error. The only valid comparison uses the shared median R^2 axis: HypatiaX Hybrid DeFi achieves median $R^2 = 1.0000$ on our 30-equation subset vs. SNIP's 0.882 on the full 119-equation SRBench set (different subsets; comparison is indicative only). Table **??** provides this comparison.

Appendix E. Hyperparameter Settings

Table 17: PySR Configuration for Symbolic Regression

Parameter	**Value**
Population size	100
Generations	500
Populations (parallel)	2 (hardware limit: 2-core CPU)
Crossover probability	0.8
Mutation probability	0.1
Complexity penalty	0.01
Tournament size	3
Elitism	5
Max expression depth	15
Optimization iterations	50
Validation Thresholds	
Acceptance R^2 (Criterion 1)	0.99
Validation score (Criterion 1)	30
Acceptance R^2 (Criterion 2)	0.95
Validation score (Criterion 2)	80
Extrapolation R^2	0.80
Numerical stability (α)	0.95

Table 18: Complete Hyperparameter Specifications for All Methods

Method	**Parameter**	**Value**
Neural Network	Architecture	[64, 32, 1]
	Activation	ReLU
	Optimizer	Adam
	Learning rate	0.01
	Batch size	Full batch (160 samples)
	Epochs	200
	Weight initialization	Xavier uniform
	Dropout	None
Pure LLM	Model	claude-sonnet-4-20250514
	Temperature	0.0 (deterministic)
	Max tokens	1024
	Top-p	1.0
	Few-shot examples	2-3 per domain
	Prompt template	Domain-specific
	Retry on failure	3 attempts
Hybrid v50_2	SR iterations	50
	SR populations	2 (hardware limit: 2-core CPU)
	Binary operators	[+, -, *, /]
	Unary operators	[exp, log, sqrt, sin, cos]
	Parsimony coefficient	0.01
	Tournament size	10
	Mutation rate	0.1
	Crossover rate	0.9
	LLM candidates	5
	LLM temperature	0.3
	Max complexity	30
	Validation threshold (R^2)	0.99

Appendix F. Detailed Experimental Results

F.1 Pure LLM Baseline (40 tests)

Table 19: Pure LLM Performance by Test Case

Domain	Test	R^2	Result	Failure Mode
Classical Scientific Domains (20 tests)				
Materials	Stress-strain	1.00	Pass	—
Materials	Young's modulus	1.00	Pass	—
Materials	Poisson ratio	1.00	Pass	—
Materials	Thermal expansion	1.00	Pass	—
Fluid	Reynolds number	1.00	Pass	—
Fluid	Bernoulli equation	1.00	Pass	—
Fluid	Drag coefficient	1.00	Pass	—
Fluid	Flow rate	1.00	Pass	—
Thermo	Ideal gas law	1.00	Pass	—
Thermo	Heat capacity	1.00	Pass	—
Thermo	Carnot efficiency	1.00	Pass	—
Thermo	Stefan-Boltzmann	1.00	Pass	—
Mechanics	Newton's 2nd law	1.00	Pass	—
Mechanics	Kinetic energy	1.00	Pass	—
Mechanics	Gravitational force	1.00	Pass	—
Mechanics	Hooke's law	1.00	Pass	—
Chemistry	Arrhenius equation	1.00	Pass	—
Chemistry	pH calculation	1.00	Pass	—
Chemistry	Nernst equation	0.50	Fail	Wrong sign convention
Chemistry	Rate law	1.00	Pass	—
Decentralized Finance Domains (20 tests)				
AMM	Constant product	1.00	Pass	—
AMM	Price impact	0.98	Pass	—
AMM	Impermanent loss	−1.26	Fail	Unit inconsistency (missing ×100)
AMM	Swap fee	0.82	Pass	—
VaR	Parametric VaR 95%	−10.05	Fail	Wrong sign (z-score)
VaR	Parametric VaR 99%	1.00	Pass	—
VaR	Historical VaR	0.95	Pass	—
VaR	Modified VaR	−2.15	Fail	Scipy API misuse
Liquidity	APY calculation	1.00	Pass	—
Liquidity	Capital efficiency	0.87	Pass	—
Liquidity	Fee earnings	—	Fail	Non-executable (tutorial mode)
Liquidity	Utilization ratio	—	Fail	Non-executable (tutorial mode)
ES	Expected Shortfall 95%	−4.12	Fail	Scipy `norm.pdf` returns object
ES	Expected Shortfall 99%	−3.87	Fail	Distribution tail error
ES	Conditional VaR	0.88	Pass	—
ES	Tail risk	−1.92	Fail	Statistical definition error
Liquidation	Long position	—	Fail	Tutorial text only
Liquidation	Short position	—	Fail	Tutorial text only
Liquidation	Margin call	—	Fail	Incomplete derivation
Liquidation	Leverage ratio	—	Fail	Multi-step missing
Classical Average:			**19/20 (95%)**	
DeFi Average:			**8/20 (40%)**	
Overall:			**27/40 (67.5%)**	

Key Findings: Pure LLM performance strongly correlates with equation prevalence in training data (Spearman $\rho = 0.89$, $p < 0.001$)(**?**). Classical equations achieve 95% success rate while specialised domains (DeFi) drop to 40%, consistent with the reproductive behaviour characterisation in Finding **??**.

F.2 DeFi-Optimized Results (23 tests)

Table 20: DeFi-Optimized HypatiaX Performance

Test	R^2	**Validation**	**Time (s)**	**Extrap.**	**Result**
Constant product (xy=k)	1.0000	100.0	45	Pass	Pass
Price impact	0.9998	95.2	120	Pass	Pass
Impermanent loss	0.9995	92.1	180	Pass	Pass
Swap fee calculation	1.0000	98.5	60	Pass	Pass
VaR 95% parametric	0.9988	88.3	150	Pass	Pass
VaR 99% parametric	0.9990	90.1	145	Pass	Pass
Historical VaR	0.9985	87.5	135	Pass	Pass
Modified VaR	0.9982	85.9	160	Pass	Pass
APY simple	1.0000	100.0	50	Pass	Pass
APY compound	0.9995	94.8	110	Pass	Pass
Capital efficiency	0.9940	78.5	240	Pass	Pass
Fee earnings	0.9998	96.2	90	Pass	Pass
Expected Shortfall 95%	0.9987	89.7	170	Pass	Pass
Expected Shortfall 99%	0.9985	88.2	175	Pass	Pass
Conditional VaR	0.9990	91.4	155	Pass	Pass
Tail risk metric	0.9983	86.8	165	Pass	Pass
Liquidation (long)	0.9992	93.6	125	—	Pass
Liquidation (short)	0.9994	94.1	120	—	Pass
Margin call threshold	0.9996	95.8	105	—	Pass
Effective leverage	0.9998	97.2	85	—	Pass
Staking rewards	1.0000	100.0	55	—	Pass
Dilution effect	0.9993	93.8	115	—	Pass
Kelly criterion	0.9988	89.5	195	Pass	Pass
Average	**0.9990**	**91.3**	**115.2**	**16/16**	**23/23**

Extrapolation Tests: Sixteen tests designated for extrapolation validation all passed with average extrapolation $R^2 = 0.9650$, indicating reliable generalization to the tested novel regimes on this benchmark(Balestriero et al., 2021; Neyshabur et al., 2017; Zhang et al., 2021).

F.3 Complete Test Suite Details

Table 21: Ground Truth Formulas with Exact Constants

Equation	Formula	Constants
Arrhenius	$k = A\exp(-E_a/(RT))$	$A = 10^{11}$, $E_a = 80000$ J/mol, $R = 8.314$ J/(mol·K)
Henderson-Hasselbalch	$\text{pH} = \text{p}K_a + \log_{10}([A^-]/[\text{HA}])$	$\text{p}K_a = 6.5$
Rate Law	$r = k[A]^2[B]$	$k = 0.5$ M^{-2}s^{-1}
Allometric Scaling	$Y = aM^b$	$a = 3.5$, $b = 0.75$
Michaelis-Menten	$v = V_{\max}[S]/(K_m + [S])$	$V_{\max} = 50$ μM/s, $K_m = 10$ μM
Logistic Growth	$dN/dt = rN(1 - N/K)$	$r = 0.3$ day^{-1}, $K = 1000$
Kinetic Energy	$E = \frac{1}{2}mv^2$	Dimensionless
Gravitational Force	$F = Gm_1m_2/r^2$	$G = 6.674 \times 10^{-11}$ N·m^2/kg^2
Ideal Gas Law	$P = nRT/V$	$R = 8.314$ J/(mol·K)
Impermanent Loss	$\text{IL} = 2\sqrt{r}/(1 + r) - 1$	Dimensionless
Price Impact	$\Delta p = \Delta x/(x + \Delta x)$	Dimensionless
Constant Product	$y = k/x$	$k = 10^6$
VaR 95%	$\text{VaR} = P\sigma \cdot z_{0.95}$	$z_{0.95} = 1.645$
Liquidation Price	$p_{\text{liq}} = p_0(1 - 1/(L \cdot m))$	$m = 0.8$ (maintenance margin)
Portfolio Variance	$\sigma_p = \sqrt{\sigma_1^2 + \sigma_2^2 + 2\rho\sigma_1\sigma_2}$	Dimensionless

Appendix G. Statistical Test Details

G.1 Mann-Whitney U Test for Extrapolation Performance

For medium extrapolation regime (2× training range):

Test Setup:

- **Sample sizes**: $n_{\text{Hybrid}} = 14$, $n_{\text{NN}} = 13$ (CORRECTED)
- **Null hypothesis**: $H_0 : E_{\text{hybrid}} \geq E_{\text{NN}}$ (Hybrid is not better)
- **Alternative hypothesis**: $H_1 : E_{\text{hybrid}} < E_{\text{NN}}$ (Hybrid is better)
- **Significance level**: $\alpha = 0.05$ (one-tailed test)

Descriptive Statistics:

- **Hybrid v50_2**: Mean = 0.0%, Median = 0.0%, SD = 0.0%, Range = [0.0, 0.0]
- **Neural Network**: Mean = 1231%, Median = 86.7%, SD = 1842%, Range = [12.3, 5847%] (CORRECTED)

Test Results:

- **Test statistic**: $U = 0$ (all Hybrid errors ¡ all NN errors)
- **P-value**: $p = 1.11 \times 10^{-6}$ (CORRECTED)
- **Critical value**: $U_{0.05} = 45$ for one-tailed test with n=14 vs n=13
- **Conclusion**: $U < U_{0.05} \Rightarrow$ Reject H_0 at $\alpha = 0.05$
- **Interpretation**: Complete rank separation — on every tested equation, Hybrid v50_2 extrapolation error is strictly less than Neural Network error ($U = 0$, $p = 1.11 \times 10^{-6}$)

G.2 Effect Size Calculation (Cohen's d)

$$\mu_{\text{NN}} = 1231.0\%, \quad \sigma_{\text{NN}} = 1842.0\% \quad \text{(CORRECTED)} \tag{1}$$

$$\mu_{\text{Hybrid}} = 0.0\%, \quad \sigma_{\text{Hybrid}} = 0.0\% \tag{2}$$

$$s_{\text{pooled}} = \sqrt{\frac{\sigma_{\text{NN}}^2 + \sigma_{\text{Hybrid}}^2}{2}} = 1302.5\% \tag{3}$$

$$d = \frac{1231.0 - 0.0}{1302.5} = 0.95 \tag{4}$$

Conservative Estimate: Using only NN standard deviation (most conservative approach):

$$d_{\text{conservative}} = \frac{1231.0}{1842.0} = 0.67 \quad \text{(medium-to-large effect)} \tag{5}$$

Alternative Calculation (Glass's Δ): Using only the control group (NN) standard deviation:

$$\Delta = \frac{1231.0 - 0.0}{1842.0} = 0.67 \tag{6}$$

Interpretation Guidelines:

- $d = 0.2$: Small effect
- $d = 0.5$: Medium effect
- $d = 0.8$: Large effect
- $d = 1.2$: Very large effect
- Our result: $d = 0.67 - 0.95$ (medium-to-large effect with full rank separation on this benchmark)

Critical Note on Effect Size: While the calculated Cohen's d appears moderate (0.67-0.95), this understates the true difference because:

1. Mann-Whitney $U = 0$ indicates full rank separation—every Hybrid error is strictly less than every NN error
2. Effect size calculations assume normal distributions, but NN errors are highly skewed (median = 86.7%, mean = 1231%)
3. The practical difference (near-zero vs 1,231% mean error on this benchmark) is substantial and potentially critical for scientific applications

G.3 Interpolation Performance Comparison

Kruskal-Wallis H Test:

- **Test statistic**: $H = 42.3$
- **P-value**: $p < 0.001$
- **Interpretation**: Significant differences exist across methods

Table 22: Interpolation Performance Statistics by System

System	n	Mean R^2	Median R^2	SD	Range
System 3 (LLM+Fallback)	30	1.000	1.000	0.000	[1.00, 1.00]
Hybrid v50_2	15	0.931	1.000	0.147	[0.58, 1.00]
Neural Network	15	0.940	0.952	0.082	[0.78, 1.00]
Pure PySR	18	0.982	0.998	0.045	[0.88, 1.00]

System Comparisons:

Key Observation: System 3 achieves $R^2 = 1.000$ on interpolation (n=30) but was not designed for extrapolation testing. Hybrid v50_2's median (1.000) exceeds its mean (0.931) due to outlier presence, making median more representative of typical performance.

G.4 Domain-Specific Success Rates

Success rate defined as $R^2 > 0.95$:

Table 23: Success Rate by Domain and Method

Method	Physics	Chemistry	Biology	DeFi	Economics
Hybrid v50_2	100%	100%	67%	100%	67%
Pure PySR	100%	100%	100%	67%	67%
LLM-Guided PySR	100%	67%	67%	67%	67%
Neural Network	100%	67%	33%	67%	33%
Pure LLM	100%	33%	0%	33%	33%
Avg across methods	100%	73%	53%	67%	53%

Correlation Analysis: Pure LLM success rate strongly correlates with equation prevalence in web text (Spearman $\rho = 0.89$, $p < 0.001$)(**?**). This is consistent with the reproductive behaviour hypothesis: performance is higher where training-corpus exposure is greater.

G.5 Validation Layer Effectiveness

Error detection breakdown across four-layer validation framework:

Table 24: Validation Layer Error Detection Statistics

Layer	**Errors**	**%**	**Cum.**	**Example**
1. Dimensional Analysis	18	12	12	Energy = mass
2. Physics Constraints	27	18	30	Negative rate
3. Numerical Stability	35	23	53	Overflow
4. Prediction Accuracy	70	47	100	Wrong result
Total	**150**	**100**	—	—

Key Finding: Single-layer validation (prediction accuracy only) would miss 53% of errors caught by earlier layers on this benchmark, indicating that cascading validation contributed substantially to error detection for LLM-generated outputs.

Appendix H. Discovered Expression Examples

H.1 Perfect Recoveries

H.1.1 Kinetic Energy

Example 1 (Kinetic Energy Discovery) ***Ground truth:*** $E = \frac{1}{2}mv^2$
HypatiaX v50_2 discovered:

$$E = ((v \times m) \times v) \times 0.5 = 0.5mv^2 \tag{7}$$

Analysis: *Exact recovery of ground truth. Discovered expression is algebraically identical.*
Interpolation: $R^2 = 1.0000$ *on training data (v ∈ [1, 10] m/s)*
Extrapolation: *0.0% error at v = 50 m/s (5× training max)*
Physical interpretation: *Correctly captures quadratic dependence on velocity and linear dependence on mass.*

H.1.2 Ideal Gas Law

Example 2 (Ideal Gas Law Discovery) ***Ground truth:*** $P = nRT/V$, *where* $R = 8.314\ J/(mol{\cdot}K)$
HypatiaX v50_2 discovered:

$$P = n \times (T \times (8.314/V)) \tag{8}$$

Analysis: *Exact recovery including the universal gas constant* $R = 8.314$ *to full precision.*
Interpolation: $R^2 = 1.0000$ *on training data*
Extrapolation: *0.0% error across all pressure/temperature/volume ranges tested (up to 5× training range)*
Physical interpretation: *On this test case, symbolic regression recovered a constant consistent with the gravitational constant from data alone, without prior knowledge of its value(**?**).*

H.2 Near-Perfect Recoveries

The Michaelis-Menten and Impermanent Loss equations were recovered with constant error $< 10^{-5}$, achieving $R^2 = 1.0000$ and near-zero extrapolation error (see Listing **??**).

H.3 Failure Case Analysis

H.3.1 Gravitational Force (Both Methods Failed)

Example 3 (Gravitational Force Failure) ***Ground truth:*** $F = Gm_1m_2/r^2$, *where* $G = 6.674 \times 10^{-11}\ N{\cdot}m^2/kg^2$
Neural Network Result:

- ***Interpolation:*** $R^2 = 0.21$ *(poor fit; relative error* $> 100\%$*, meeting the threshold in Definition **??**)*
- ***Extrapolation:*** *Not attempted due to interpolation failure*

- ***Issue**: Gradient vanishing due to 10^{-11} scale*
- ***Attempted solution**: Log-transform → marginal improvement ($R^2 = 0.35$)*
- ***Root cause**: Optimization landscapes too flat at this scale for gradient descent*

***HypatiaX v50_2 Result**:*

- ***Interpolation**: $R^2 = -0.03$ (worse than baseline)*
- ***Extrapolation**: Not attempted due to interpolation failure*
- ***Discovered form**: $F = 1.23 \times 10^{13} \frac{m_1 m_2}{r^2}$ (correct functional form, wrong constant by $2 \times 10^{23}\times$)*
- ***Issue**: PySR constant search range $[10^{-3}, 10^{3}]$ missed $G \sim 10^{-11}$*
- ***Attempted solution**: Extend range to $[10^{-15}, 10^{15}] \rightarrow$ no convergence in 50 iterations*
- ***Root cause**: Search space too large for evolutionary algorithm to explore efficiently*

***Recommended Solution**:*

1. ***Dimensional analysis preprocessing**: Identify required constant scales before symbolic regression*
2. ***Logarithmic transformation**: Convert $F = Gm_1m_2/r^2$ to $\log F = \log G + \log m_1 + \log m_2 - 2\log r$, making the problem linear in log-space*
3. ***Adaptive constant ranges**: Dynamically adjust search based on data magnitude*
4. ***Multi-scale normalization**: Normalize inputs/outputs to [0.1, 10] range before discovery, then rescale*

***Lesson learned**: Very small constants ($< 10^{-6}$) require special handling in both neural and symbolic methods. This represents 1/15 test cases (6.7% failure rate) and is a known limitation addressed in Section 12.*

H.3.2 Henderson-Hasselbalch (Pure LLM Failure)

Example 4 (Henderson-Hasselbalch pH Calculation) ***Ground truth**: $pH = pKa + \log_{10}\left(\frac{[A^-]}{[HA]}\right)$*

***Pure LLM Generated**:*

$$pH = pKa \times \frac{[A^-]}{[HA]} \tag{9}$$

***Failure Analysis**:*

- ***R^2 score**: -12.3 (predictions anticorrelated with ground truth)*
- ***Errors**:*
  1. *Multiplication instead of addition*

2. *Missing logarithm entirely*
3. *Plausible but fundamentally wrong*

- ***Root cause**: LLM training data likely contains more qualitative discussions of pH ("pH increases when conjugate base increases") than quantitative formulas*
- ***Pattern matching failure**: LLM defaulted to simpler multiplicative relationship rather than logarithmic form*

***Symbolic Rescue**: PySR correctly discovered logarithmic relationship:*

$$pH = pKa + \log_{10}\left(\frac{[A^-]}{[HA]}\right) \tag{10}$$

Achieving $R^2 = 0.998$, demonstrating the value of hybrid architecture.

***Lesson learned**: LLM performance declined on this specialised equation, which is consistent with Finding **??**—generation probability is theoretically predicted to be lower for expressions that are structurally distant from the training corpus(**?**).*

H.4 Complete Discovered Expressions Code Listing

```
# Kinetic Energy (PERFECT)
discovered: ((v * m) * v) * 0.5
ground_truth: 0.5 * m * v**2
R2: 1.0000, extrapolation_error: 0.0%

# Rate Law (PERFECT)
discovered: ((A_conc * 0.5) * A_conc) * B_conc
ground_truth: 0.5 * A_conc**2 * B_conc
R2: 1.0000, extrapolation_error: 0.0%

# Ideal Gas Law (PERFECT)
discovered: n * (T * (8.314 / V))
ground_truth: n * 8.314 * T / V
R2: 1.0000, extrapolation_error: 0.0%

# Logistic Growth (PERFECT - algebraically equivalent)
discovered: ((N * -0.0003) + 0.3) * N
ground_truth: 0.3 * N * (1 - N/1000)
expanded: 0.3*N - 0.0003*N^2 (identical)
R2: 1.0000, extrapolation_error: 0.0%

# Price Impact (PERFECT)
discovered: swap / (swap + reserve)
ground_truth: dx / (x + dx)
R2: 1.0000, extrapolation_error: 0.0%

# Michaelis-Menten (NEAR-PERFECT)
discovered: (S / (S + 10.000007)) * 50.000008
ground_truth: 50 * S / (10 + S)
```

```
constant_error: <1e-5
R2: 1.0000, extrapolation_error: 0.0%

# Impermanent Loss (NEAR-PERFECT)
discovered: (2 * sqrt(r)) / (1 + r) - 1.000023
ground_truth: 2 * sqrt(r) / (1 + r) - 1
constant_error: 2.3e-5
R2: 1.0000, extrapolation_error: 0.0%
```

Listing 1: Discovered Symbolic Expressions (7 Perfect Recoveries)

Appendix I. Supplementary Materials

The following supplementary materials are available:

- **S1: Pure LLM Baseline Results** (`S1_pure_llm_results.csv`) — Complete results from Pure LLM baseline experiments showing 60% success rate across 40 tests.
- **S2: Extrapolation Test Data** (`S2_extrapolation_data.csv`) — Data comparing Hybrid v50_2 extrapolation error (near-zero) versus Neural Network extrapolation error (1,231% mean) on the Core 15 benchmark. Includes training R^2 scores, extrapolation errors at 1.2×, 2×, and 5× training ranges, and Mann-Whitney U-test results (U=0, $p<10^{-6}$).
- **S3: Validation Logs** (`S3_validation_logs.json`) — Complete validation reports for all 131 tests showing the four-layer validation framework in action. Each entry includes dimensional analysis, physics constraints, numerical stability checks, prediction accuracy metrics, timing data, and failure mode classification. Includes validation layer rejection rates (12%, 18%, 23%, 47%).
- **S4: Discovered Symbolic Expressions** (`S4_discovered_expressions.{txt,json}`) — All 131 discovered symbolic expressions in SymPy-compatible format, organized by domain (chemistry, biology, physics, DeFi, finance) with complete metadata including ground truth equations, performance metrics (R^2, timing, iterations), and algebraic equivalence verification.
- **S5: Timing Breakdown** (`S5_timing_breakdown.csv`) — Detailed computational cost analysis showing component-level timing: LLM initialization (0-39%), symbolic search (50-91%), and validation (7-11%). Demonstrates the 4.3× speedup from LLM-guided initialization versus pure symbolic methods.
- **S6: Domain-Specific Analysis** (`S6_domain_analysis.xlsx`) — Statistical analysis by scientific domain showing success rates: Biology 100%, Chemistry 100%, Physics 100%, DeFi 100%, Finance 77.8%. Includes per-equation breakdowns and domain-specific constraint validation results.
- **S7: Failure Mode Taxonomy** (`S7_failure_mode_taxonomy.pdf`) — Comprehensive taxonomy of 23 distinct failure modes categorized by method (Pure LLM: 8 modes,

Neural Network: 3 modes, Symbolic: 5 modes, Hybrid: 2 modes, Integration: 5 modes), severity (8 Critical, 4 High, 9 Medium, 2 Low), and detection layer. Includes detailed examples of silent semantic errors, large-scale extrapolation errors, distributional reasoning failures, and API misuse.

- **S8: Figure Source Data** (`S8_figures_source_data/`) — Source data files for all figures in the paper:
  - `figure1_arrhenius_extrapolation.csv` — Arrhenius equation extrapolation comparison
  - `figure2_domain_comparison.csv` — Success rates by scientific domain
  - `figure3_validation_breakdown.csv` — Four-layer validation effectiveness
  - `figure4b_domain_breakdown.csv` — Per-equation performance details
  - `figure5_method_comparison.csv` — Overall method comparison
  - `figure_5systems_comparison.csv` — Complete extrapolation data (1231% vs 0%)
  - `figure_benchmark_comparison.csv` — Core 15 benchmark results
  - `figure_timing_comparison.csv` — Speed-accuracy tradeoff analysis

All supplementary materials are provided in both synthetic versions (matching reported statistics exactly) and real experimental versions (direct from JSON logs) for maximum reproducibility and transparency. Complete documentation and usage examples are provided in accompanying README files.

I.1 Supplementary Data Files

All supplementary data are available at:

`https://github.com/sednabcn/LLM-HypatiaX-PAPERS`

Table 25: Supplementary Data Files

File	Contents
`S1_pure_llm_results.csv`	Complete Pure LLM test results (n=40) with formulas, R^2 scores, failure modes
`S2_extrapolation_data.csv`	Extrapolation test data ($n = 27$) at 1.2×, 2×, 5× training ranges
`S3_validation_logs.json`	Full validation reports for all 131 tests
`S4_discovered_expressions.txt`	All discovered symbolic expressions in SymPy format
`S5_timing_breakdown.csv`	Detailed timing data by component and method
`S6_domain_analysis.xlsx`	Per-domain performance with statistical tests
`S7_failure_taxonomy.pdf`	Complete taxonomy of 23 failure modes with examples
`S8_figures_source_data/`	Raw data for all 8 figures in CSV format

I.2 Reproducibility Materials

Complete step-by-step tutorials for reproducing all experiments are available at: `https://sednabcn.github.io/ai-llm-blog/tutorials/hypatiax/`

- Tutorial 1: Environment Setup and First Discovery (15 min)
- Tutorial 2: Running Benchmark Experiments (45 min active + 3–8 hours compute)
- Tutorial 3: Statistical Analysis and Publication Figures (30 min)
- Tutorial 4: Custom Applications and Extensions (45 min)

All source code and data generation scripts are available at: `https://github.com/sednabcn/LLM-HypatiaX-PAPERS`

I.3 Reproducibility Artifacts

Docker Images:

- `hypatiax:v1.0` - Full environment with all dependencies
- `hypatiax:minimal` - Core symbolic regression only (no LLM)
- `hypatiax:gpu` - GPU-accelerated version for neural network comparison

Source Code: All Python scripts and experimental protocols are available at:

`https://github.com/sednabcn/LLM-HypatiaX-PAPERS/tree/master/papers/2025-JMLR/hypatiax`

Jupyter Notebooks:

- `01_data_generation.ipynb` - Synthetic data generation for 15 equations
- `02_pure_llm_experiments.ipynb` - Pure LLM baseline (40 tests)
- `03_hybrid_experiments.ipynb` - Hybrid v50_2 experiments (30 tests)
- `04_extrapolation_analysis.ipynb` - Extrapolation tests and statistical analysis
- `05_figure_generation.ipynb` - Reproduction of all 8 figures
- `06_statistical_tests.ipynb` - Mann-Whitney U, Kruskal-Wallis, effect sizes

All Python Jupyter notebooks are available at:

`https://github.com/sednabcn/LLM-HypatiaX-PAPERS/tree/master/papers/2025-JMLR/hypatiax/notebooks`

Step-by-Step Tutorials:

- Tutorial 1: Environment Setup and First Discovery (15 min)
- Tutorial 2: Running Benchmark Experiments (45 min active + 3–8 hours compute)
- Tutorial 3: Statistical Analysis and Publication Figures (30 min)
- Tutorial 4: Custom Applications and Extensions (45 min)

Available at: `https://sednabcn.github.io/ai-llm-blog/tutorials/hypatiax/`

I.4 Additional Statistical Tests

Table 26: Additional Statistical Tests

Comparison	Test	Result	Interpretation
Hybrid v50_2 vs Pure PySR	Mann-Whitney	$U = 89$, $p = 0.18$	No significant difference
System 3 vs Hybrid v50_2	Mann-Whitney	$U = 156$, $p = 0.03$	System 3 better (interp.)
Classical vs DeFi (LLM)	Fisher's Exact	$p < 0.001$	Domain effect significant
Run-to-run variability	Friedman	$\chi^2 = 34.2$, $p < 0.001$	High variability

Power Analysis:

- Extrapolation comparison (n=14 vs n=13): Power = 1.000 for d ¿ 0.5
- Domain comparisons (n=3 per domain): Power = 0.65 for d ¿ 0.8
- Method comparisons (n=15 per method): Power = 0.95 for d ¿ 0.6

Recommendation: Future work should increase per-domain sample sizes to n $\geq$ 10 for robust inference on secondary effects.

Appendix J. Noise Sensitivity Analysis

To assess correspondence with Condition 3 of Empirical Result **??** (noise tolerance threshold $\sigma \leq 0.1 \cdot \text{std}(y)$), we conducted systematic experiments varying noise levels.

J.1 Experimental Protocol

We tested 15 classical formulas (Table **??**) with noise levels:

$$\sigma \in \{0.01, 0.03, 0.05, 0.08, 0.10, 0.15, 0.20\} \times \text{std}(y)$$

For each noise level, we:

1. Generated $n = 100$ training points with additive Gaussian noise: $y_{\text{noisy}} = y_{\text{true}} + \mathcal{N}(0, \sigma^2)$
2. Ran PySR with standard hyperparameters (Appendix E)
3. Evaluated success as $R^2 \geq 0.99$ and correct functional form
4. Repeated 5 times per configuration to account for evolutionary randomness

J.2 Results

Table 27: Success Rate vs Noise Level

Noise Level (σ/std(y))	**Success Rate**	**95% CI**
0.01	100% (75/75)	[95.2%, 100%]
0.03	97.3% (73/75)	[90.7%, 99.7%]
0.05	93.3% (70/75)	[85.1%, 97.8%]
0.08	78.7% (59/75)	[67.7%, 87.3%]
0.10	66.7% (50/75)	[54.8%, 77.1%]
0.15	45.3% (34/75)	[33.8%, 57.3%]
0.20	24.0% (18/75)	[14.9%, 35.3%]

J.3 Key Findings

Sharp Threshold at $\sigma = 0.1$: Success rate drops from 93.3% ($\sigma = 0.05$) to 45.3% ($\sigma = 0.15$), consistent with the empirically derived threshold. Fisher's exact test shows a significant difference between $\sigma \leq 0.10$ and $\sigma > 0.10$ ($p < 0.001$).

Domain Variability: Classical mechanics formulas (kinetic energy, potential energy) tolerate higher noise ($\sigma \leq 0.15$) while chemistry formulas (Arrhenius, rate law) fail above $\sigma = 0.08$. This suggests domain-specific complexity affects noise sensitivity.

Failure Modes: Above threshold, common failures include:

- Incorrect exponents (e.g., $v^{1.8}$ instead of v^2 for kinetic energy)
- Spurious additive constants to compensate for noise
- Overfitting with complex expressions ($R^2 > 0.99$ on training, poor extrapolation)

J.4 Implications

Our main experiments use $\sigma = 0.05 \cdot \text{std}(y)$, safely below the threshold, ensuring 93.3% baseline success rate from noise alone. Real-world applications should:

1. Estimate noise level from repeated measurements
2. Apply denoising if $\sigma > 0.08 \cdot \text{std}(y)$
3. Increase sample size by factor of $(\sigma/0.05)^2$ to maintain success rate